

Contents

0.1	Validation & Benchmarking Suite	9
0.2	Validation & Benchmarking Suite (Kernel Validation Layer) . . .	9
0.2.1	13.0 Purpose	9
0.2.2	13.1 Validation Concepts	9
0.2.3	13.2 Core Validation Requirements	10
0.2.4	13.3 Noise & Perturbation Robustness	10
0.2.5	13.4 Adversarial Structured Input Tests	11
0.2.6	13.5 Parameter Perturbation Tests	11
0.2.7	13.6 Memory Load & Schema Stress Tests	11
0.2.8	13.7 Multi-Lattice Conflict Tests	12
0.2.9	13.8 Breathing-Self-Organization Synchronization	12
0.2.10	13.9 SES Evolution Tests	12
0.2.11	13.10 Cross-Domain Transfer Tests	12
0.2.12	13.11 Global Stability Shock Tests	13
0.2.13	13.12 Validation Summary	13
0.3	Section 23 — Hardening Framework	37
0.3.1	23.0 Hardening Overview	37
0.3.2	23.1 Hardening Trigger Conditions (HTCs)	38
0.3.3	23.2 Hardening Actions (HAs)	39
0.3.4	23.3 SafeMode	40
0.3.5	23.4 Hardening Recovery Rules (HRRs)	40
0.3.6	23.5 Traceability	41
0.3.7	23.6 Summary	41

L0 — Structural Field Normalization

Purpose

L0 converts raw input streams into a mathematically well-posed Normalized Structural Field (NSF). It enforces signal integrity, derives structural quantities, and ensures deterministic behavior for all downstream layers.

Architecture Context

L0 is the first stage of the UF kernel pipeline:

$$F(t) \rightarrow \text{SEV}(t) \rightarrow \text{NSF}$$

L1 gate formation depends entirely on correct L0 outputs.

Inputs

Raw time-indexed field:

$$F : T \rightarrow \mathbb{R}^d$$

Outputs

Normalized Structural Field:

$$\text{NSF} = \{\text{SEV}(t) \mid t \in T\}$$

Input Health Verification

L0 rejects input if:

- Timestamp monotonicity fails
- $\|\Delta F(t)\| > \Delta_{\max}$
- $\sigma(t) \notin [0, \sigma_{\max})$
- $F(t) \notin D \subset \mathbb{R}^d$

Derived Structural Quantities

$$\Delta F(t) = F(t) - F(t - \Delta t)$$

$$\sigma(t) = \frac{1}{W} \sum_{i=0}^{W-1} \|F(t - i\Delta t) - \bar{F}_W(t)\|^2$$

$$\kappa(t) = \|F(t + \Delta t) - 2F(t) + F(t - \Delta t)\|$$

Negative-Space Operator

$$N(t) = \begin{cases} 1 & \sigma(t) < \sigma_{\min} \wedge \|\Delta F(t)\| < \delta_{\min} \wedge \kappa(t) < \kappa_{\min} \\ 0 & \text{otherwise} \end{cases}$$

State Embedding Vector

$$\text{SEV}(t) = (F(t), \Delta F(t), \sigma(t), \kappa(t), r(t), N(t))$$

Perturbation Stability

$$\|SEV(F + \varepsilon) - SEV(F)\| \leq C\|\varepsilon\|$$

Role in Kernel Pipeline

L0 guarantees downstream determinism for gate formation, mosaic projection, and schema evolution.

L1 — Gate and Mosaic Architecture

Purpose

L1 segments the NSF into gates and embeds them into one or more mosaic lattices for structural classification.

Architecture Context

L1 consumes L0 outputs:

$$NSF \rightarrow \{G_k, TVR_k, P_\ell(G_k), C_k\}$$

Gate Boundary Operator

$$D(t) = \alpha_1 \|\Delta F(t)\| + \alpha_2 \sigma(t) + \alpha_3 \kappa(t)$$
$$D(t) \geq \tau_D \Rightarrow \text{boundary}$$

Gate Definition

A gate $G_k = [t_a, t_b]$ satisfies:

$$\forall t \in (t_a, t_b) : D(t) < \tau_D$$

Time–Volume–Relevance Vector

$$TVR_k = (T_k, V_k, R_k)$$
$$T_k = t_b - t_a$$
$$V_k = \int_{t_a}^{t_b} (\beta_1 \|\Delta F\| + \beta_2 \sigma + \beta_3 \kappa) dt$$
$$R_k = \int_{t_a}^{t_b} r(t) dt$$

Mosaic Projection Operator

$$P_\ell(G_k) = \left(\left\lfloor \frac{T_k}{h_1^{(\ell)}} \right\rfloor, \left\lfloor \frac{V_k}{h_2^{(\ell)}} \right\rfloor, \left\lfloor \frac{R_k}{h_3^{(\ell)}} \right\rfloor \right)$$

Mosaic Divergence

$$C_k = |\{P_1(G_k), \dots, P_L(G_k)\}|$$

Gate Drift

$$\delta_g(G_k) = \|\mu_k - \mu_{k-1}\|$$

Role in Kernel Pipeline

L1 determines the structural segmentation and quantization that all downstream interpretive, resonance, and decision mechanisms depend on.

L2 — Interpretive Structural Metrics

Purpose

L2 transforms gate-level descriptors produced by L1 into structurally meaningful interpretive metrics. These metrics quantify intensity, contrast, stability, structural uncertainty, regime classification, and anomaly detection for downstream resonance and decision dynamics.

Architecture Context

L2 consumes L1 outputs:

$$\{G_k, TVR_k, P_\ell(G_k), C_k, \delta_g(G_k), N(G_k)\}$$

and produces:

$$ISF_k = (w_k, CV_k, S_k, Reg_k, U_k, IAS_k)$$

Inputs

- Gate structure: $G_k = [t_a, t_b]$
- Time–Volume–Relevance vector: $TVR_k = (T_k, V_k, R_k)$
- Mosaic projections: $P_\ell(G_k)$
- Divergence: C_k
- Gate drift: $\delta_g(G_k)$
- Negative-space gate flag: $N(G_k)$

Outputs

Interpretive Structural Field (ISF):

$$ISF_k = (w_k, CV_k, S_k, Reg_k, U_k, IAS_k)$$

Interpretive Weight

$$w_k \in [0, 1]$$

represents normalized structural intensity derived from average first difference, variance, and curvature within G_k .

Contrast Vector

$$CV_k = TVR_k - \mu_k$$

where μ_k is the local structural baseline across neighboring gates.

Structural Score

$$S_k = \gamma_1 w_k + \gamma_2 \frac{\|CV_k\|}{\|CV\|_{\max}} + \gamma_3 \frac{1}{1 + C_k}$$

Regime Classification

Given structural density $\chi_k = V_k/T_k$ and normalized contrast $\psi_k = \|CV_k\|/\|CV\|_{\max}$:

- Stable: $\chi_k < \chi_{\min}$ and $\psi_k < \psi_{\min}$
- Transitional: $\chi_{\min} \leq \chi_k \leq \chi_{\max}$
- Volatile: $\chi_k > \chi_{\max}$
- Degenerate: $\psi_k > \psi_{\max}$

Uncertainty

$$U_k = \lambda_1 \frac{C_k - 1}{L - 1} + \lambda_2 \frac{\delta_g(G_k)}{\delta_{\max}} + \lambda_3 N(G_k)$$

Interpretive Anomaly Suppression

$$IAS_k = 1 \iff U_k > U_{\max}$$

Role in Kernel Pipeline

L2 provides the structural metrics that determine resonance alignment, anomaly suppression, and decision dynamics.

L3 — Resonance Engine

Purpose

L3 aggregates interpretive metrics into a single scalar resonance measure $R(k)$. Resonance quantifies structural alignment, stability, and interpretive coherence for each gate.

Architecture Context

L3 consumes L2 outputs:

$$(w_k, CV_k, S_k, Reg_k, U_k, IAS_k)$$

and produces:

$$URF_k = g_k \cdot R(k)$$

Resonance Vector

$$R_k^{vec} = \left(w_k, \frac{\|CV_k\|}{\|CV\|_{\max}}, S_k, \frac{1}{1 + C_k}, 1 - U_k \right)$$

Resonance Functional

$$R(k) = \frac{1}{Z} \left(\lambda_1 w_k + \lambda_2 \frac{\|CV_k\|}{\|CV\|_{\max}} + \lambda_3 S_k + \lambda_4 \frac{1}{1 + C_k} + \lambda_5 (1 - U_k) \right)$$

with $Z = \sum_i \lambda_i$.

Resonance Hysteresis

$$Hyst_k = 1 \iff |R(k) - R(k-1)| > h_{\max}$$

Contextual Gating

$$g_k = 1 \iff (U_k \leq U_{\max}) \wedge (IAS_k = 0) \wedge (Hyst_k = 0)$$

Unified Resonance Field

$$URF_k = g_k \cdot R(k)$$

Role in Kernel Pipeline

L3 provides the scalar resonance signal that informs decision dynamics (L4), ensuring structural consistency and stability across interpretive transitions.

L4 — Decision Dynamics

Purpose

L4 transforms resonance signals from L3 into directional, momentum, reversal, uncertainty-adjusted, and breathing-aware components that form the Decision Surface Vector (DSF). The DSF is the sole interface from the kernel to L5 domain adapters.

Architecture Context

L4 consumes L3 outputs:

$$URF_k = g_k \cdot R(k)$$

and produces:

$$DSF_k = (D_k, M_k, R_k, U_k^*, C_k, P_k, B_k)$$

Directional Signal

Let $\Delta R(k) = R(k) - R(k-1)$. Define:

$$D_k = \begin{cases} +1 & \Delta R(k) > \epsilon_D \\ 0 & |\Delta R(k)| \leq \epsilon_D \\ -1 & \Delta R(k) < -\epsilon_D \end{cases}$$

Momentum and Reversal

$$M_k = R(k) - 2R(k-1) + R(k-2)$$

$$R_k = 1 \iff D_k \cdot D_{k-1} < 0$$

Adjusted Uncertainty

$$U_k^* = U_k + \eta_H Hyst_k + \eta_{IAS} IAS_k$$

Pressure Metric

$$P_k = |D_k - D_{k-1}|$$

Breathing Dynamics

$$B_k = B_{k-1} + \xi(1 - U_k^*)\Delta R(k) - \chi U_k^*$$

with clamping:

$$B_k = \max(B_{\min}, \min(B_{\max}, B_k))$$

Decision Surface Vector

$$DSF_k = (D_k, M_k, R_k, U_k^*, C_k, P_k, B_k)$$

Role in Kernel Pipeline

L4 determines structural direction, stability, interpretive load, and momentum supplied to L5, while preserving kernel invariants.

Section 11 — Formal Mathematical Framework

Purpose

This section defines the mathematical operators, constraints, and invariants that govern the Unified Framework. All other sections must be consistent with these definitions.

State Embedding Vector

$$SEV(t) = (F(t), \Delta F(t), \sigma(t), \kappa(t), r(t), N(t))$$

Perturbation stability:

$$\|SEV(F + \varepsilon) - SEV(F)\| \leq C_{SEV} \|\varepsilon\|.$$

Gate Boundary Operator

$$D(t) = \alpha_1 \|\Delta F(t)\| + \alpha_2 \sigma(t) + \alpha_3 \kappa(t)$$

Gate opens when:

$$D(t) < \tau_D.$$

Mosaic Projection

$$P_\ell(G_k) = \left(\left\lfloor \frac{T_k}{h_1^{(\ell)}} \right\rfloor, \left\lfloor \frac{V_k}{h_2^{(\ell)}} \right\rfloor, \left\lfloor \frac{R_k}{h_3^{(\ell)}} \right\rfloor \right)$$

Divergence:

$$C_k = |\{P_1(G_k), \dots, P_L(G_k)\}|.$$

Contrast Vector

$$CV_k = TVR_k - \mu_k.$$

Resonance Functional

$$R(k) = \frac{1}{Z} \left(\lambda_1 w_k + \lambda_2 \frac{\|CV_k\|}{\|CV\|_{\max}} + \lambda_3 S_k + \lambda_4 \frac{1}{1 + C_k} + \lambda_5 (1 - U_k) \right).$$

Hysteresis

$$Hyst_k = 1 \iff |R(k) - R(k-1)| > h_{\max}.$$

Decision Surface Formalism

$$DSF_k = (D_k, M_k, R_k, U_k^*, C_k, P_k, B_k).$$

Global Stability Functional

$$S(UF) = \lambda_c(1 - Coh) + \lambda_h(1 - H_s) + \lambda_b(1 - Var(B)) + \lambda_u(1 - \bar{U}) + \lambda_d(1 - Drift(T)).$$

Robustness Index

$$R(UF) = 1 - \frac{1}{K} \sum_{i=1}^K |S(UF) - S(UF^{(i)})|.$$

Invariants

$$0 \leq R(k) \leq 1, \quad 0 \leq U_k \leq 1, \quad C_k \leq C_{\max}, \quad H_s \leq H_{\max}, \quad B_{\min} \leq B_k \leq B_{\max}.$$

SafeMode triggers when:

$$S(UF) < S_{\min}.$$

0.1 Validation & Benchmarking Suite

0.2 Validation & Benchmarking Suite (Kernel Validation Layer)

0.2.1 13.0 Purpose

This section defines deterministic validation procedures ensuring Unified Framework (UF) structural correctness, robustness, bounded schema growth, mosaic consistency, SES compatibility, and shock recovery.

0.2.2 13.1 Validation Concepts

13.1.1 Global Stability Functional $S(UF)$

The global stability functional is defined as:

$$S(UF) = \lambda_c(1 - Coh) + \lambda_h(1 - H_s) + \lambda_b(1 - Var(B)) + \lambda_u(1 - \bar{U}) + \lambda_d(1 - Drift(T)). \quad (1)$$

All terms are normalized into the range $[0, 1]$.

13.1.2 Robustness Index $R(UF)$

For a set of deterministic perturbation trials $\{UF^{(i)}\}_{i=1}^K$, the robustness index is:

$$R(UF) = 1 - \frac{1}{K} \sum_{i=1}^K |S(UF) - S(UF^{(i)})|. \quad (2)$$

13.1.3 Structural Validation Domains

Validation is performed across multiple structural domains, including:

- Low-structure fields (smooth, low-variance signals),
- High-structure fields (rapidly varying, high-curvature signals),
- Adversarial structural fields (synthetically constructed to maximize divergence, drift, or variance).

0.2.3 13.2 Core Validation Requirements

The UF must satisfy the following structural requirements under all tested conditions:

- **DSF bounded variation:**

$$\|\text{DSF}_k - \text{DSF}_{k-1}\| \leq \varepsilon_{\text{DSF}} \quad \text{unless} \quad |\Delta R(k)| > \varepsilon_D. \quad (3)$$

- **Schema entropy bound:**

$$H_s \leq H_{\max}. \quad (4)$$

- **Mosaic divergence bound:**

$$C_k \leq C_{\max}. \quad (5)$$

- **Cross-layer consistency:**

$$\text{XLC}(k) = 1 \quad \text{for all } k. \quad (6)$$

- **Breathing stability:**

$$\text{Var}(B) \leq \varepsilon_B. \quad (7)$$

- **No catastrophic failure:** Under perturbation, the UF must not diverge, collapse, explode schema memory, or break mosaic consistency.

0.2.4 13.3 Noise & Perturbation Robustness

For a baseline field $F_0(t)$ and bounded perturbation pattern $u(t)$ with $\|u(t)\| \leq 1$, noisy inputs are given by:

$$F_\varepsilon(t) = F_0(t) + \varepsilon u(t). \quad (8)$$

The UF must satisfy:

- **Stability deviation:**

$$\Delta S(\varepsilon) = |S(\text{UF}_0) - S(\text{UF}_\varepsilon)| \leq \kappa_S \varepsilon. \quad (9)$$

- **Uncertainty deviation:**

$$\Delta U(\varepsilon) \leq \kappa_U \varepsilon . \quad (10)$$

- **Breathing stability:**

$$\text{Var}(B_\varepsilon) \leq \text{Var}(B_0) + \kappa_B \varepsilon . \quad (11)$$

0.2.5 13.4 Adversarial Structured Input Tests

Adversarial classes (A–E) are synthetically constructed to maximize:

- structural drift,
- mosaic divergence,
- curvature extremes,
- variance spikes.

Pass criteria require:

$$\Delta S_X \leq \kappa_{\text{ADV}} \varepsilon_X , \quad (12)$$

$$U_X \leq U_{\max} , \quad (13)$$

$$C_k(X) \leq C_{\max} , \quad (14)$$

$$S_{\text{count}} \leq S_{\max} , \quad (15)$$

$$0 \leq R(k) \leq 1 , \quad (16)$$

$$B_k \in [B_{\min}, B_{\max}] , \quad (17)$$

$$\text{negative-space usage remains within structural limits.} \quad (18)$$

0.2.6 13.5 Parameter Perturbation Tests

For parameter set Θ and perturbed parameters $\Theta' = \Theta + \Delta\Theta$ with $\|\Delta\Theta\| \leq \varepsilon_\Theta$, the UF must satisfy:

$$\Delta S_\Theta = |S(\text{UF}_\Theta) - S(\text{UF}_{\Theta'})| \leq \kappa_{S\Theta} \varepsilon_\Theta , \quad (19)$$

and must preserve bounds on schema entropy, mosaic divergence, breathing variance, and drift.

0.2.7 13.6 Memory Load & Schema Stress Tests

Long-sequence structural variation is used to test schema stability. The UF must satisfy:

$$S_{\text{count}} \leq S_{\max} , \quad (20)$$

$$H_s \leq H_{\max} , \quad (21)$$

$$\text{Drift}(T) \leq D_{\max} , \quad (22)$$

$$C_k \leq C_{\max} \quad \forall k , \quad (23)$$

$$S(\text{UF}, k) \geq S_{\min} \quad \forall k . \quad (24)$$

0.2.8 13.7 Multi-Lattice Conflict Tests

Inputs are constructed to create deliberate conflicts across mosaic projections. The UF must maintain:

$$C_k \leq C_{\max} \quad \forall k, \quad (25)$$

$$U_k \leq U_{\max} \quad \forall k, \quad (26)$$

$$0 \leq R(k) \leq 1, \quad (27)$$

$$S_{\text{count}} \leq S_{\max}, \quad (28)$$

$$S(\text{UF}, k) \geq S_{\min} \quad \forall k. \quad (29)$$

0.2.9 13.8 Breathing–Self-Organization Synchronization

Regime A/B scenarios test the coupling between breathing amplitude and schema mutation rate. The UF must satisfy:

$$A_B(k) \leq \alpha_1 D_T(k) + \alpha_2 |\Delta R(k)| + B_0, \quad (30)$$

$$\mu_s(k) \leq \mu_{s,\max}, \quad (31)$$

$$A_B(k) \mu_s(k) \leq \beta_{\max}, \quad (32)$$

$$S(\text{UF}, k) \geq S_{\min} \quad \forall k. \quad (33)$$

0.2.10 13.9 SES Evolution Tests

For SES versions parameterized by Θ and Θ' (with bounded parameter differences), SES must remain compatible:

$$\text{StructEq}(\Gamma_k, \Gamma'_k) = 1, \quad (34)$$

$$\Delta K_k \leq \kappa_K \varepsilon_{\Theta}, \quad (35)$$

$$S(\text{UF}_{\Theta'}) \geq S_{\min}, \quad (36)$$

where Γ_k and Γ'_k are the original and decrypted structural tuples.

0.2.11 13.10 Cross-Domain Transfer Tests

Across structural domains of dimension d_X , the UF must maintain:

$$\Delta S_X \leq \kappa_{SD} d_X, \quad (37)$$

$$C_k \leq C_{\max} \quad \forall k, \quad (38)$$

$$H_s \leq H_{\max}, \quad (39)$$

$$\text{Drift}(T_X) \leq D_{\max, X}, \quad (40)$$

$$\|\text{DSF}_k\| \leq D_{\max}, \quad (41)$$

$$\text{Var}(B_X) \leq \text{Var}(B_0) + \kappa_{BD} d_X. \quad (42)$$

0.2.12 13.11 Global Stability Shock Tests

Shock types include lattice break, resonance spikes, catastrophic variance, extreme parameter drift, and structural regime breaks. Under shocks, UF must satisfy:

$$S(\text{UF}_{\text{shock}}) \geq 0, \quad (43)$$

$$S(\text{UF}_{\text{shock}}) < S_{\min} \iff \text{SafeMode} = 1, \quad (44)$$

$$C_k \leq C_{\max} \quad \forall k, \quad (45)$$

$$S_{\text{count}} \leq S_{\max}, \quad (46)$$

$$B_k \in [B_{\min}, B_{\max}] \quad \forall k, \quad (47)$$

and after removal of the shock,

$$\lim_{t \rightarrow t_r} S(\text{UF}(t)) = S(\text{UF}_0). \quad (48)$$

0.2.13 13.12 Validation Summary

The Unified Framework (UF) is considered validated only if all structural, robustness, SES, mosaic, schema, breathing, and shock-related criteria defined in this section are simultaneously satisfied across all tested domains, perturbations, and scenarios.

Executable Validation Protocols

Test 13-1 — Noise Robustness Executable Protocol

Goal: Verify that the UF engine behaves predictably when small bounded noise is added to the baseline input field.

Preconditions

- Python 3.9+ is installed.
- Folder structure exists:

```
UF-Engine/
  uf_engine.py
  config.yaml
  inputs/
  outputs/
  logs/
  scripts/
```

- Baseline structural field exists at:

```
inputs/baseline.json
```

- Noise generator script exists at:

`scripts/generate_noise.py`

- `config.yaml` from the UF-Spec v1.4.0 baseline is present.

Required Data

- Baseline input file:

`inputs/baseline.json`

- Noise levels:

$$\varepsilon \in \{0.01, 0.05, 0.10, 0.20, 0.30\}$$

Setup

Open a terminal and run these commands from inside `UF-Engine/`:

```
python scripts/generate_noise.py --epsilon 0.01 --input inputs/baseline.json --output inputs/test_13-1_noise_0.01.json
python scripts/generate_noise.py --epsilon 0.05 --input inputs/baseline.json --output inputs/test_13-1_noise_0.05.json
python scripts/generate_noise.py --epsilon 0.10 --input inputs/baseline.json --output inputs/test_13-1_noise_0.10.json
python scripts/generate_noise.py --epsilon 0.20 --input inputs/baseline.json --output inputs/test_13-1_noise_0.20.json
python scripts/generate_noise.py --epsilon 0.30 --input inputs/baseline.json --output inputs/test_13-1_noise_0.30.json
```

This creates five noisy input files in `inputs/`.

Execution Steps

1. Run the UF engine on the baseline file:

```
python uf_engine.py --config config.yaml \
--input inputs/baseline.json \
--output outputs/baseline_result.json \
--log logs/baseline.log
```

2. Run the UF engine on each noisy input file:

```
python uf_engine.py --config config.yaml \
--input inputs/test_13-1_noise_0.01.json \
--output outputs/test_13-1_noise_0.01_result.json \
--log logs/test_13-1_noise_0.01.log
```

```
python uf_engine.py --config config.yaml \
--input inputs/test_13-1_noise_0.05.json \
--output outputs/test_13-1_noise_0.05_result.json \
--log logs/test_13-1_noise_0.05.log
```

```
python uf_engine.py --config config.yaml \
--input inputs/test_13-1_noise_0.10.json \
--output outputs/test_13-1_noise_0.10_result.json \
--log logs/test_13-1_noise_0.10.log
```

```
python uf_engine.py --config config.yaml \
--input inputs/test_13-1_noise_0.20.json \
--log logs/test_13-1_noise_0.20.log
```

```

--output outputs/test_13-1_noise_0.20_result.json \
--log logs/test_13-1_noise_0.20.log

python uf_engine.py --config config.yaml \
--input inputs/test_13-1_noise_0.30.json \
--output outputs/test_13-1_noise_0.30_result.json \
--log logs/test_13-1_noise_0.30.log

```

3. For each output file in `outputs/`, open the JSON and locate:

- "delta_S"
- "delta_U"
- "delta_Var_B"
- "invariant_violations"

Expected Results

For each noise level:

- "invariant_violations" must be an empty list:

[]
- "delta_S", "delta_U", and "delta_Var_B" must be small non-negative numbers that gently increase with ε .

Pass / Fail Criteria

PASS if all noise levels satisfy:

- "invariant_violations" is empty.
- "delta_S" $\leq \kappa_S \varepsilon$
- "delta_U" $\leq \kappa_U \varepsilon$
- "delta_Var_B" $\leq \kappa_B \varepsilon$

FAIL if any noise level shows:

- A non-empty "invariant_violations" list,
- A large unexpected spike in "delta_S", "delta_U", or "delta_Var_B".

Traceability

- Stability functional $S(\text{UF})$ (Section 13.1)
- Noise robustness bounds (Section 13.3)
- Hardening & SafeMode conditions (Section 23)
- L0–L4 operational equations (Part I)

Test 13-2 — Adversarial Structured Input Protocol

Goal: Verify that the UF engine maintains structural invariants and bounded behavior when exposed to adversarially constructed input fields designed to maximize drift, curvature, variance, and mosaic divergence.

Preconditions

- Python 3.9+ is installed.
- The UF-Engine folder structure exists:

```
UF-Engine/  
  uf_engine.py  
  config.yaml  
  inputs/  
  outputs/  
  logs/  
  scripts/
```

- Baseline field exists at:

 - Adversarial generator script exists at:

 scripts/generate_adversarial.py
- config.yaml from UF-Spec v1.4.0 is present.

Required Data

Adversarial classes A–E must be generated. Each class targets a specific structural weakness.

- Class A: maximize drift
- Class B: maximize curvature
- Class C: maximize variance
- Class D: maximize mosaic divergence
- Class E: combined adversarial pattern

Generated files will follow this naming pattern:

```
inputs/test_13-2_adv_A.json  
inputs/test_13-2_adv_B.json  
inputs/test_13-2_adv_C.json  
inputs/test_13-2_adv_D.json  
inputs/test_13-2_adv_E.json
```


Setup

Use the adversarial generator script to create inputs.

Run these commands from inside **UF-Engine/**:

```
python scripts/generate_adversarial.py --mode A --output inputs/test_13-2_adv_A.json
python scripts/generate_adversarial.py --mode B --output inputs/test_13-2_adv_B.json
python scripts/generate_adversarial.py --mode C --output inputs/test_13-2_adv_C.json
python scripts/generate_adversarial.py --mode D --output inputs/test_13-2_adv_D.json
python scripts/generate_adversarial.py --mode E --output inputs/test_13-2_adv_E.json
```

This creates the adversarial inputs used in this test.

Execution Steps

1. Run the UF engine on each adversarial input:

```
python uf_engine.py --config config.yaml \
--input inputs/test_13-2_adv_A.json \
--output outputs/test_13-2_adv_A_result.json \
--log logs/test_13-2_adv_A.log

python uf_engine.py --config config.yaml \
--input inputs/test_13-2_adv_B.json \
--output outputs/test_13-2_adv_B_result.json \
--log logs/test_13-2_adv_B.log

python uf_engine.py --config config.yaml \
--input inputs/test_13-2_adv_C.json \
--output outputs/test_13-2_adv_C_result.json \
--log logs/test_13-2_adv_C.log

python uf_engine.py --config config.yaml \
--input inputs/test_13-2_adv_D.json \
--output outputs/test_13-2_adv_D_result.json \
--log logs/test_13-2_adv_D.log

python uf_engine.py --config config.yaml \
--input inputs/test_13-2_adv_E.json \
--output outputs/test_13-2_adv_E_result.json \
--log logs/test_13-2_adv_E.log
```

2. For each result file in **outputs/**, open the JSON and locate:

- "delta_S"
- "delta_U"
- "delta_Var_B"
- "invariant_violations"
- "dsf_summary"

Expected Results

For each adversarial class (A–E):

- No crashes or halts.

- "invariant_violations" must remain:

[]

- Breathing values must remain within:

$$B_{\min} \leq B_k \leq B_{\max}$$

- Mosaic divergence must satisfy:

$$C_k \leq C_{\max}$$

- Uncertainty must satisfy:

$$U_k \leq U_{\max}$$

- DSF components must remain finite and bounded.

Pass / Fail Criteria

PASS if all adversarial cases satisfy:

- "invariant_violations" is empty
- "delta_S" shows smooth deviation, not spikes
- "delta_U" within expected bounds
- "delta_Var_B" within expected bounds
- No unexpected SafeMode activation

FAIL if ANY of the following occur:

- Any non-empty "invariant_violations"
- Breathing out of bounds
- Mosaic divergence beyond $C_{\max}$
- Uncertainty beyond $U_{\max}$
- DSF blowup (NaN or INF)
- Unexpected SafeMode activation

Traceability

- Section 13.4 Adversarial Input Requirements
- Section 11 Kernel Invariants
- Section 23 Hardening Framework
- L1/L2 Mosaic & Interpretive Metrics
- L4 DSF Stability

Test 13-3 — Parameter Perturbation Protocol

Goal: Verify that small, bounded changes to UF kernel parameters do not cause instability, invariant violations, or discontinuities in UF processing.

Preconditions

- Python 3.9+ is installed.
- The UF-Engine folder structure exists:

```
UF-Engine/  
  uf_engine.py  
  config.yaml  
  config_variants/  
  inputs/  
  outputs/  
  logs/
```

- Baseline config exists:

```
config.yaml
```

- Parameter variants folder exists:

```
config_variants/
```

- Baseline input exists:

```
inputs/baseline.json
```

Required Data

Create parameter-perturbed config files with controlled deviations.

Example perturbation set:

- `config_var_1.yaml` — small increase in τ_D
- `config_var_2.yaml` — small decrease in $\psi_{\max}$
- `config_var_3.yaml` — perturb λ_i resonance weights
- `config_var_4.yaml` — perturb breathing parameters $B_{\min}, B_{\max}$
- `config_var_5.yaml` — perturb uncertainty weights $\lambda_1, \lambda_2, \lambda_3$

All changes must satisfy:

$$\|\Delta\Theta\| \leq \varepsilon_{\Theta}$$

Setup

Place all variant configs here:

```
config_variants/  
  config_var_1.yaml  
  config_var_2.yaml  
  config_var_3.yaml  
  config_var_4.yaml  
  config_var_5.yaml
```

Execution Steps

1. Run baseline:

```
python uf_engine.py --config config.yaml \  
  --input inputs/baseline.json \  
  --output outputs/baseline_param_result.json \  
  --log logs/baseline_param.log
```

2. For each variant config file, run:

```
python uf_engine.py --config config_variants/config_var_1.yaml \  
  --input inputs/baseline.json \  
  --output outputs/test_13-3_var1_result.json \  
  --log logs/test_13-3_var1.log
```

```
python uf_engine.py --config config_variants/config_var_2.yaml \  
  --input inputs/baseline.json \  
  --output outputs/test_13-3_var2_result.json \  
  --log logs/test_13-3_var2.log
```

```
python uf_engine.py --config config_variants/config_var_3.yaml \  
  --input inputs/baseline.json \  
  --output outputs/test_13-3_var3_result.json \  
  --log logs/test_13-3_var3.log
```

```
python uf_engine.py --config config_variants/config_var_4.yaml \  
  --input inputs/baseline.json \  
  --output outputs/test_13-3_var4_result.json \  
  --log logs/test_13-3_var4.log
```

```
python uf_engine.py --config config_variants/config_var_5.yaml \  
  --input inputs/baseline.json \  
  --output outputs/test_13-3_var5_result.json \  
  --log logs/test_13-3_var5.log
```

3. For each output file, open:

- "delta_S"
- "delta_U"
- "delta_Var_B"
- "invariant_violations"
- "dsf_summary"

Expected Results

For all parameter variants:

- No crashes or UF halts.
- "invariant_violations" is always:

$\square$

- DSF components remain bounded.
- Breathing stays within:

$$B_{\min} \leq B_k \leq B_{\max}$$

- Mosaic divergence stays within:

$$C_k \leq C_{\max}$$

- Uncertainty stays within:

$$U_k \leq U_{\max}$$

Pass / Fail Criteria

PASS if all variant configs satisfy:

- "invariant_violations" is empty
- "delta_S" $\leq \kappa_{S\Theta}\varepsilon_\Theta$
- "delta_U" $\leq \kappa_{S\Theta}\varepsilon_\Theta$
- "delta_Var_B" within structural expectation

FAIL if ANY variant config shows:

- Non-empty invariants violations
- Extreme breathing deviation
- Mosaic divergence out of bounds
- Uncertainty out of bounds
- DSF blowup (NaN/INF)

Traceability

- Section 13.5 Parameter Perturbation Requirements
- Section 11 Kernel Invariants
- Section 23 Hardening Constraints
- L0–L4 Operational Equations

Test 13-4 — Memory & Schema Stress Protocol

Goal: Verify that the UF engine maintains bounded schema growth, stable drift behavior, and invariant satisfaction when processing long structural sequences with sustained variation.

Preconditions

- Python 3.9+ is installed.
- UF-Engine directory structure exists:

```
UF-Engine/  
  uf_engine.py  
  config.yaml  
  inputs/  
  outputs/  
  logs/  
  scripts/
```

- Long-run structural sequence generator exists:

```
scripts/generate_long_sequence.py
```

- config.yaml from UF-Spec v1.4.0 is present.

Required Data

Use sequence generator to create a long structural field.

Example:

```
python scripts/generate_long_sequence.py \  
--length 5000 \  
--output inputs/test_13-4_longseq.json
```

The resulting file should contain:

- 5,000+ timestamps
- smooth and rough structural segments mixed
- multiple regime transitions
- sparse negative-space intervals

Execution Steps

1. Run UF engine on long sequence:

```
python uf_engine.py --config config.yaml \  
--input inputs/test_13-4_longseq.json \  
--output outputs/test_13-4_longseq_result.json \  
--log logs/test_13-4_longseq.log
```

2. Open the output file and check the following fields:

- "schema_count"
- "max_C_k"
- "max_U_k"
- "max_B_k", "min_B_k"
- "drift_measure"
- "invariant_violations"

Expected Results

UF must maintain stable memory and schema behavior:

- Schema count:

$$S_{\text{count}} \leq S_{\text{max}}$$

- Schema entropy:

$$H_s \leq H_{\text{max}}$$

- Mosaic divergence:

$$C_k \leq C_{\text{max}}$$

- Drift:

$$\text{Drift}(T) \leq D_{\text{max}}$$

- Breathing:

$$B_{\text{min}} \leq B_k \leq B_{\text{max}}$$

- No invariant violations.

Pass / Fail Criteria

PASS if all of the following hold:

- "invariant_violations" is []
- Schema count does not exceed S_{max}
- Mosaic divergence remains within bounds

- Drift remains below $D_{\max}$
- Breathing stays within allowed range
- No NaNs or INF values in DSF summary

FAIL if ANY of the following occur:

- Schema count runaway
- Schema entropy runaway
- Breathing runaway or collapse
- Drift spikes
- Mosaic divergence out of bounds
- Any invariant violation

Traceability

- Section 13.6 Memory & Schema Stress Requirements
- Section 11 Kernel Invariants
- Section 23 Hardening Constraints
- L1–L4 structural segmentation, interpretation, resonance, and DSF

Test 13-5 — Multi-Lattice Mosaic Conflict Protocol

Goal: Verify that the UF engine maintains stable mosaic divergence, bounded uncertainty, and valid DSF behavior when lattice projections disagree (conflicting mosaic embeddings).

Preconditions

- Python 3.9+ is installed.
- UF-Engine directory structure exists:

```
UF-Engine/
  uf_engine.py
  config.yaml
  inputs/
  outputs/
  logs/
  scripts/
```

- Mosaic conflict generator script exists:

`scripts/generate_mosaic_conflict.py`

- `config.yaml` from UF-Spec v1.4.0 is present.

Required Data

Generate mosaic-conflict inputs designed to cause disagreement across mosaics:

```
python scripts/generate_mosaic_conflict.py \  
--output inputs/test_13-5_conflict.json
```

Generated file must include:

- structural segments projected differently across lattices
- mixed curvature patterns
- alternating expansion/compression regions
- at least one section where mosaic classes disagree

Execution Steps

1. Run UF engine on the mosaic-conflict input:

```
python uf_engine.py --config config.yaml \  
--input inputs/test_13-5_conflict.json \  
--output outputs/test_13-5_conflict_result.json \  
--log logs/test_13-5_conflict.log
```

2. Open the result file and check:

- `"max_C_k"` (maximum mosaic divergence)
- `"max_U_k"` (maximum uncertainty)
- `"max_B_k"` and `"min_B_k"` (breathing)
- `"invariant_violations"`
- `"dsf_summary"`

Expected Results

- Mosaic divergence must remain within:

$$C_k \leq C_{\max}$$

- Uncertainty must satisfy:

$$U_k \leq U_{\max}$$

- Breathing must satisfy:

$$B_{\min} \leq B_k \leq B_{\max}$$

- "invariant_violations" must be:

[]

- Resonance must remain finite and bounded.

Pass / Fail Criteria

PASS if:

- Mosaic divergence remains within $C_{\max}$
- Uncertainty remains within $U_{\max}$
- Breathing remains within allowed bounds
- No invariant violations
- DSF remains finite (no NaN/INF)

FAIL if ANY occur:

- $C_k > C_{\max}$
- $U_k > U_{\max}$
- Breathing out of range
- Any invariant violation
- DSF instability (blowup)

Traceability

- Section 13.7 Multi-Lattice Conflict Requirements
- Section 11 Kernel Invariants
- Section 23 Hardening Framework
- L1 Mosaic Projection Operators
- L2 Uncertainty Metrics

Test 13-6 — Breathing & Self-Organization Synchronization Protocol

Goal: Verify that breathing dynamics (bandwidth modulation) remain synchronized with schema evolution under different structural regimes (Regime A and Regime B), without causing runaway growth or instability.

Preconditions

- Python 3.9+ is installed.
- UF-Engine directory structure exists:

```
UF-Engine/  
  uf_engine.py  
  config.yaml  
  inputs/  
  outputs/  
  logs/  
  scripts/
```

- Regime scenario generator script exists:

```
scripts/generate_regime_scenarios.py
```

- config.yaml from UF-Spec v1.4.0 is present.

Required Data

Generate two structural scenarios:

- Regime A: high activity (higher drift, more variation)
- Regime B: low activity (lower drift, more stable)

Using:

```
python scripts/generate_regime_scenarios.py \  
  --mode A --output inputs/test_13-6_regimeA.json
```

```
python scripts/generate_regime_scenarios.py \  
  --mode B --output inputs/test_13-6_regimeB.json
```

Execution Steps

1. Run UF on Regime A:

```
python uf_engine.py --config config.yaml \  
  --input inputs/test_13-6_regimeA.json \  
  --output outputs/test_13-6_regimeA_result.json \  
  --log logs/test_13-6_regimeA.log
```

2. Run UF on Regime B:

```
python uf_engine.py --config config.yaml \
--input inputs/test_13-6_regimeB.json \
--output outputs/test_13-6_regimeB_result.json \
--log logs/test_13-6_regimeB.log
```

3. For each result, check the following fields:

- "breathing_amplitude_series" or summary fields:
 - "max_A_B"
 - "mean_A_B"
- "schema_mutation_rate"
- "drift_measure"
- "invariant_violations"

Expected Results

Under Regime A (higher structural activity):

- Breathing amplitude should be higher relative to Regime B.
- Schema mutation rate should increase but stay bounded.

Under Regime B (lower structural activity):

- Breathing amplitude should contract relative to Regime A.
- Schema mutation rate should decrease.

In both regimes:

- No invariant violations.
- Breathing amplitude and schema mutation rate must remain within configured bounds.

Pass / Fail Criteria

PASS if:

- Regime A:
 - Higher breathing amplitude than Regime B
 - Higher schema mutation rate than Regime B
- Regime B:
 - Lower breathing amplitude than Regime A
 - Lower schema mutation rate than Regime A

- In both:
 - No invariant violations
 - No runaway increase in breathing or schema count

FAIL if:

- Breathing or schema mutation run away in either regime
- Any invariant violation occurs
- Regime A/B behavior does not show expected relative differences

Traceability

- Section 13.8 Breathing–Self-Organization Synchronization
- Section 11 Kernel Invariants
- Section 19 Self-Organization Dynamics
- Section 23 Hardening Framework

Test 13-7 — SES Evolution Protocol

Goal: Verify that SES (Structural Encryption System) remains structurally compatible and invertible under bounded changes to SES-related parameters, and across UF/SES version transitions.

Preconditions

- Python 3.9+ is installed.
- UF-Engine directory structure exists:

```
UF-Engine/
  uf_engine.py
  config.yaml
  ses_config/
  inputs/
  outputs/
  logs/
  scripts/
```

- SES-compatible UF engine is implemented: encryption/decryption paths active.
- SES configuration files exist in:

```
ses_config/
  ses_v1.yaml
  ses_v2.yaml
```

- Baseline structural input exists:

`inputs/baseline.json`

Required Data

Two SES configuration versions:

- `ses_v1.yaml` — original SES parameters
- `ses_v2.yaml` — updated SES parameters with:
 - bounded changes to masks
 - bounded changes to permutation indices
 - bounded changes to shift vectors

All SES parameter changes must satisfy:

$$\|\Delta\Theta_{\text{SES}}\| \leq \varepsilon_{\Theta}$$

Execution Steps

1. Encrypt and decrypt using SES v1:

```
python uf_engine.py --config config.yaml \
--ses-config ses_config/ses_v1.yaml \
--input inputs/baseline.json \
--output outputs/test_13-7_ses_v1_result.json \
--log logs/test_13-7_ses_v1.log
```

2. Encrypt and decrypt using SES v2:

```
python uf_engine.py --config config.yaml \
--ses-config ses_config/ses_v2.yaml \
--input inputs/baseline.json \
--output outputs/test_13-7_ses_v2_result.json \
--log logs/test_13-7_ses_v2.log
```

3. For each SES version output, examine:

- `"structural.equivalence"` (boolean)
- `"delta_K"` (key drift measure)
- `"invariant.violations"`
- `"ses.status"` (e.g., `"ok"` or `"error"`)

Expected Results

- For SES v1 and v2:
 - Structural equivalence must be preserved:

$$\text{StructEq}(\Gamma_k, \Gamma'_k) = 1$$

- Key drift must satisfy:

$$\Delta K_k \leq \kappa_K \varepsilon_\Theta$$

- No invariant violations.
 - SES status reports "ok".

Pass / Fail Criteria

PASS if:

- "structural_equivalence" is true for both SES versions
- "delta_K" is within $\kappa_K \varepsilon_\Theta$
- No invariant violations
- SES status is "ok"

FAIL if ANY occur:

- Structural equivalence fails
- Key drift exceeds bound
- Any invariant violation
- SES error status

Traceability

- Section 13.9 SES Evolution Requirements
- Appendix C — SES Encryption Platform
- Section 11 Kernel Invariants
- Section 23 Hardening & Compatibility Constraints

Test 13-8 — Cross-Domain Transfer Protocol

Goal: Verify that the UF engine maintains stable behavior, bounded invariants, and valid DSF outputs when the same structural patterns are embedded into multiple domains of different dimensionality and scaling.

Preconditions

- Python 3.9+ is installed.
- UF-Engine directory structure exists:

```
UF-Engine/  
  uf_engine.py  
  config.yaml  
  inputs/  
  outputs/  
  logs/  
  scripts/
```

- Cross-domain generator script exists:

```
scripts/generate_cross_domain.py
```

- config.yaml from UF-Spec v1.4.0 is present.

Required Data

Generate structurally similar inputs for multiple domains:

- Domain 1: $d_1 = 1$
- Domain 2: $d_2 = 3$
- Domain 3: $d_3 = 5$

Use:

```
python scripts/generate_cross_domain.py --dim 1 --output inputs/test_13-8_dim1.json  
python scripts/generate_cross_domain.py --dim 3 --output inputs/test_13-8_dim3.json  
python scripts/generate_cross_domain.py --dim 5 --output inputs/test_13-8_dim5.json
```

Execution Steps

1. Run UF engine on each domain input:

```
python uf_engine.py --config config.yaml \  
  --input inputs/test_13-8_dim1.json \  
  --output outputs/test_13-8_dim1_result.json \  
  --log logs/test_13-8_dim1.log  
  
python uf_engine.py --config config.yaml \  
  --input inputs/test_13-8_dim3.json \  
  --output outputs/test_13-8_dim3_result.json \  
  --log logs/test_13-8_dim3.log  
  
python uf_engine.py --config config.yaml \  
  --input inputs/test_13-8_dim5.json \  
  --output outputs/test_13-8_dim5_result.json \  
  --log logs/test_13-8_dim5.log
```

2. For each result file, check:

- "delta_S"
- "delta_U"
- "delta_Var_B"
- "invariant_violations"
- "dsf_summary"

Expected Results

For all domain dimensions d_X tested:

- Stability deviations satisfy:

$$\Delta S_X \leq \kappa_{SD} \cdot d_X$$

- Mosaic divergence stays within:

$$C_k \leq C_{\max}$$

- Schema entropy stays within:

$$H_s \leq H_{\max}$$

- Drifts remain bounded:

$$\text{Drift}(T_X) \leq D_{\max, X}$$

- DSF values remain finite and bounded.

- Breathing variance stays within:

$$\text{Var}(B_X) \leq \text{Var}(B_0) + \kappa_{BD} \cdot d_X$$

- No invariant violations.

Pass / Fail Criteria

PASS if all domains satisfy:

- Structural deviations within $\kappa_{SD} \cdot d_X$
- Mosaic divergence within bounds
- Schema entropy within bounds
- Drift bounded
- Breathing bounded

- No invariant violations

FAIL if ANY domain shows:

- Excessive ΔS_X
- Mosaic divergence out of bounds
- Schema entropy runaway
- Drift runaway
- Breathing runaway
- Any invariant violation

Traceability

- Section 13.10 Cross-Domain Transfer Requirements
- Section 11 Kernel Invariants
- Section 23 Hardening Framework
- L0–L4 Operator Consistency

Test 13-9 — Global Stability Shock Protocol

Goal: Verify that the UF engine remains stable, obeys invariants, activates SafeMode only when appropriate, and returns to baseline stability after extreme structural shocks, including: lattice breaks, resonance spikes, catastrophic variance bursts, large parameter jumps, and structural regime breaks.

Preconditions

- Python 3.9+ is installed.
- UF-Engine directory structure exists:

```
UF-Engine/
  uf_engine.py
  config.yaml
  inputs/
  outputs/
  logs/
  scripts/
```

- Shock scenario generator exists:

```
scripts/generate_shock_scenarios.py
```

- Baseline input exists:

```
inputs/baseline.json
```

- config.yaml from UF-Spec v1.4.0 is present.

Required Data

Generate five extreme scenarios:

- Shock A: Lattice break
- Shock B: Resonance spike
- Shock C: Catastrophic variance burst
- Shock D: Extreme parameter drift
- Shock E: Structural regime break

Using:

```
python scripts/generate_shock_scenarios.py --type A --output inputs/test_13-9_shock_A.json
python scripts/generate_shock_scenarios.py --type B --output inputs/test_13-9_shock_B.json
python scripts/generate_shock_scenarios.py --type C --output inputs/test_13-9_shock_C.json
python scripts/generate_shock_scenarios.py --type D --output inputs/test_13-9_shock_D.json
python scripts/generate_shock_scenarios.py --type E --output inputs/test_13-9_shock_E.json
```

Execution Steps

1. Run UF on each shock scenario:

```
python uf_engine.py --config config.yaml \
--input inputs/test_13-9_shock_A.json \
--output outputs/test_13-9_shock_A_result.json \
--log logs/test_13-9_shock_A.log

python uf_engine.py --config config.yaml \
--input inputs/test_13-9_shock_B.json \
--output outputs/test_13-9_shock_B_result.json \
--log logs/test_13-9_shock_B.log

python uf_engine.py --config config.yaml \
--input inputs/test_13-9_shock_C.json \
--output outputs/test_13-9_shock_C_result.json \
--log logs/test_13-9_shock_C.log

python uf_engine.py --config config.yaml \
--input inputs/test_13-9_shock_D.json \
--output outputs/test_13-9_shock_D_result.json \
--log logs/test_13-9_shock_D.log

python uf_engine.py --config config.yaml \
--input inputs/test_13-9_shock_E.json \
--output outputs/test_13-9_shock_E_result.json \
--log logs/test_13-9_shock_E.log
```

2. For each output file, locate:

- "post_shock_S" — stability after shock
- "safe_mode" — boolean
- "invariant_violations"
- "drift_measure"
- "max_C_k", "max_U_k", "max_B_k"
- "recovery_S" — stability after recovery window

Expected Results

For all shock types:

- Stability immediately after shock must satisfy:

$$S(\text{UF}_{\text{shock}}) \geq 0$$

- If:

$$S(\text{UF}_{\text{shock}}) < S_{\min}$$

then SafeMode must activate (true).

- Invariants remain enforceable:

$$C_k \leq C_{\max}, \quad U_k \leq U_{\max}, \quad B_{\min} \leq B_k \leq B_{\max}$$

- After the recovery window:

$$\text{recovery_S} \rightarrow S(\text{UF}_0)$$

returning to baseline stability.

Pass / Fail Criteria

PASS if ALL conditions hold:

- Post-shock $S(\text{UF}) \geq 0$
- SafeMode activation only when $S < S_{\min}$
- Invariants remain satisfied (no violations)
- Drift remains bounded
- DSF remains finite
- Recovery stability returns near baseline

FAIL if ANY occur:

- SafeMode fails to trigger when required
- SafeMode triggers when not required
- Invariants violated
- Drift runaway
- Breathing runaway
- DSF blowup
- No return to baseline after shock removal

Traceability

- Section 13.11 Global Stability Shock Requirements
- Section 11 Kernel Invariants
- Section 23 Hardening Triggers (HTC)
- L0–L4 structural dynamics

Validation Matrix

III Test ID Description Pass Criteria Summary

13-1	Noise Robustness	$\Delta S(\epsilon) \leq \kappa_S \epsilon$	
13-2	Adversarial Inputs	All structural bounds hold	
13-3	Parameter Drift	$\Delta S_\Theta \leq \kappa_{S\Theta} \epsilon_\Theta$	
13-4	Memory Stress	$S_{\text{count}} \leq S_{\text{max}}$	
13-5	Mosaic Conflict	$C_k \leq C_{\text{max}}$	
13-6	Breathing Sync	$A_B(k)\mu_s(k) \leq \beta_{\text{max}}$	
13-7	SES Evolution	$\Delta K_k \leq \kappa_K \epsilon_\Theta$	
13-8	Cross Domain	Stability preserved across domains	
13-9	Shock Test	SafeMode triggers iff $S < S_{\text{min}}$	

0.3 Section 23 — Hardening Framework

Purpose: Define the Unified Framework (UF) safety controls that prevent structural instability, ensure bounded behavior across all kernel layers (L0–L4), enforce invariants, and direct SafeMode activation and recovery. Hardening ensures that the UF never produces runaway states, invalid outputs, or unbounded schema/mosaic dynamics.

0.3.1 23.0 Hardening Overview

Hardening consists of three components:

- **Hardening Trigger Conditions (HTCs)** — When the UF must intervene.
- **Hardening Actions (HAs)** — Exact responses taken when a trigger fires.
- **Hardening Recovery Rules (HRRs)** — How the UF returns to normal mode.

Hardening applies continuously in L0–L4 and is referenced by all Section 13 protocols.

0.3.2 23.1 Hardening Trigger Conditions (HTCs)

An HTC fires when ANY safety-critical invariant is violated or crosses its warning threshold.

HTC-1: Mosaic Divergence Violation

$$C_k > C_{\max}$$

HTC-2: Uncertainty Violation

$$U_k > U_{\max}$$

HTC-3: Breathing Range Violation

$$B_k < B_{\min} \quad \text{or} \quad B_k > B_{\max}$$

HTC-4: Schema Growth Violation

$$S_{\text{count}} > S_{\max}$$

HTC-5: Schema Entropy Violation

$$H_s > H_{\max}$$

HTC-6: Stability Collapse

$$S(\text{UF}) < S_{\min}$$

HTC-7: DSF Instability

$$\|\text{DSF}_k - \text{DSF}_{k-1}\| > \varepsilon_{\text{DSF}}$$

in the absence of a valid resonance transition ($|\Delta R(k)| \leq \varepsilon_D$).

HTC-8: Drift Violation

$$\text{Drift}(T) > D_{\max}$$

HTC-9: Resonance Blowup

$$R(k) \notin [0, 1]$$

(due to corruption or failed gating).

HTC-10: SES Decryption Failure

$$\text{StructEq}(\Gamma_k, \Gamma'_k) = 0$$

0.3.3 23.2 Hardening Actions (HAs)

When ANY HTC fires, the UF applies one or more of these deterministic actions.

HA-1: Gate Suppression

The current gate G_k is suppressed:

$$g_k = 0$$

This prevents invalid resonance or decision propagation.

HA-2: DSF Neutralization

The DSF is replaced by a neutral vector:

$$\text{DSF}_k = (0, 0, 0, 1, C_k, 0, B_{k-1})$$

This stabilizes reasoning to a “hold” position.

HA-3: Uncertainty Spike

$$U_k \leftarrow 1$$

This prevents L4 from making high-confidence transitions.

HA-4: Breathing Contraction

$$B_k \leftarrow \max(B_{\min}, B_k - \beta_B)$$

HA-5: Schema Freeze

No new schemas may form:

$$\Delta S_{\text{count}} = 0$$

HA-6: Schema Merge

If schemas are proliferating:

$$\text{merge all } C_i \text{ with } \|C_i - C_j\| < \tau_{\text{merge}}$$

HA-7: Mosaic Compression

Force mosaic projections toward consensus:

$$C_k \leftarrow 1 + \min(C_k - 1, \gamma_C)$$

HA-8: SES Failsafe Mode

Disable encryption for this cycle and emit warning:

$$\text{SES_status} = \text{"failsafe"}$$

0.3.4 23.3 SafeMode

SafeMode is a global state in which:

- All decision dynamics freeze.
- DSF is replaced by a neutral hold vector.
- Schema formation is suspended.
- Mosaic projections are clamped.
- L3 resonance gating g_k is forced to 0.
- Hardening remains active until recovery rules are met.

Formal condition:

$$\text{SafeMode} = 1 \iff S(\text{UF}) < S_{\min}$$

In SafeMode, the system may still interpret input, but cannot:

- escalate resonance
- update schemas
- expand breathing
- produce directional decisions

0.3.5 23.4 Hardening Recovery Rules (HRRs)

To exit SafeMode or normal hardening:

HRR-1: Stability Recovery

$$S(\text{UF}) \geq S_{\min}$$

HRR-2: Mosaic Recovery

$$C_k \leq C_{\max}$$

HRR-3: Breathing Recovery

$$B_{\min} \leq B_k \leq B_{\max}$$

HRR-4: Schema Recovery

$$S_{\text{count}} \leq S_{\max}$$

HRR-5: SES Recovery

$\text{SES_status} = \text{"ok"}$

When ALL HRRs are satisfied:

$\text{SafeMode} \leftarrow 0$

and all kernel layers resume normal operation.

0.3.6 23.5 Traceability

- Section 11 — Kernel Invariants
- Section 13 — Validation Requirements (all tests reference these HTCs/HAs/HRRs)
- Section 15 — SES compatibility and error handling
- Section 19 — Self-Organization limits
- Section 22 — Governance enforcement

0.3.7 23.6 Summary

Section 23 defines the precise safety boundaries for UF operation. All kernel layers (L0–L4), validation protocols, and SES integration rely on these controls. A UF instance that violates any HTC without triggering a corresponding Hardening Action or SafeMode is non-compliant with UF-Spec v1.4.0.